



Draw It or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	4
Domain Model	4
Evaluation	5
Recommendations	10

Document Revision History

Version	Date	Author	Comments
1.0	11/16/2025	Kimberly Gilbraith	Initial document creations. Documentation added for Executive summary, Requirements, Design constraints, System architecture, Domain model, Evaluation, and Recommendations.
2.0	11/29/2025	Kimberly Gilbraith	Evaluation and recommendations section updated
3.0	12/14/2025	Kimberly Gilbraith	Revisions and updates to recommendations section

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Introduction and Problem Statement

Creative Technology Solutions (CTS) is partnering with a client, The Gaming Room, to expand their Android only application into a scalable web-based games, usable across multiple platforms.

A software design challenges lies in establishing a single platform for game management accessible from multiple platforms. The application must support simultaneous user instances that span across multiple teams and platforms, ensuring unique features such as team names and session instances.

Proposed Solution

I propose leveraging a secure cloud-based data and storage service to utilize as the host for the game's application and interface. Furthermore, I propose a reversioning of current game code to move it to a platform-agnostic framework allowing for a unified code design, friendly to all web-based browsers and Android platforms. This design allows for an accelerated expansion of current design and features across desired platforms.

Requirements

To meet the needs of CTS and The Gaming room, the design must meet the following requirements:

- Cloud-based platform – The back-end application must be stored and ran from a cloud-based OS
- Single codebase – Coding should be done primarily through a platform-agnostic framework, allowing for utilization of a centralized location for instruction and management of the application
- Security and Privacy – Development must meet all regions' security, privacy and payment system requirements through secure user authentication portals and platforms.
- Unique Game elements- In order to run successfully, mechanisms need to be in place to support unique game elements to include but not limited to:
 - Game instances
 - Team Names
 - User Names
- Minimize expenditures – overall expenses should be limited until increased revenue streams arrive. This will include but not limited to
 - Hardware requirements
 - Development costs
 - Cloud infrastructure

Design Constraints

Based on the identified application requirements, the following constraints will be applied to the applications design:

- Cloud-based Infrastructure – A requirement to keep costs minimal forces a dependency on a cloud-based infrastructure, avoiding costs associated with Virtual Machines, (VMs) or physical servers.
- Single Codebase – A platform-agnostic framework needs to be utilized to enforce single instance development across multiple desktop platforms and browsers.
- Strict Data Integrity – unique elements must be upheld throughout development to ensure compliance with local and regional data regulations.

Implications for Design and Development

Each of the previous constraints translates into the following implications for the design and development team:

- Cloud-based Infrastructure – Coding must be logical and concise, minimizing server call instances, and speeding up game elements.
- Single Codebase – Testing time must increase beyond that allowed for single platform development to ensure complete testing and debugging across multiple platforms and browsers. Testing must include multiple configurations of players, browsers and platforms to ensure compatibility and response times between configurations.
- Strict Data Integrity – A trusted third-party authentication and payment system must be integrated early to ensure regulatory compliance.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The following UML Diagram for The Gaming Room's Win It or Loose IT application will serve as the basis for the code implementation and design.

- Abstract Class (Entity) serves as a root directory for all game objects. In this diagram these include the classes Game, Team, and Player.
 - Attributes in this class (id and name), ensure unique game identifiers for each instance.
 - Methods for this class include the getters, (getID() and getName()), and constructors of Entity(id: long, name: String)
 - Game Class allows for a single instance of the game.
 - Attributes in this class are inherited from Entity (id and name)
 - Methods in this class include commands to add teams to the instance (addTeam(name:String): Team)
 - Team Class allows for a unique team instance to be created.
 - Attributes in this class are inherited from Entity (id and Name)
 - Methods in this class allow players to be added to a team (addPlayer(name: String): Player)
 - Player Class allows for a unique user to be added into a game instance
 - Attributes are inherited from Entity (id and name)
- A Singleton Manager, (GameService), will be utilized to control all instances and interactions
 - Attributes create a list of game elements for game instances (nextGameID, nextPlayerID, nextTeamId) and the static instance (service: GameService)
 - Methods (addGame, getGame, getGameCount, getNextPlayerID, getNextTeamId) assist with the fundamental instance options to Create, Read, Update and Delete information.
- SingletonTester and ProgramDriver are separate classes where the ProgramDriver uses SingletonTester for execution and verification of instances.
 - ProgramManager contains the main() method to start the application
 - SingletonTester verifies only a single instance of GameServices exists

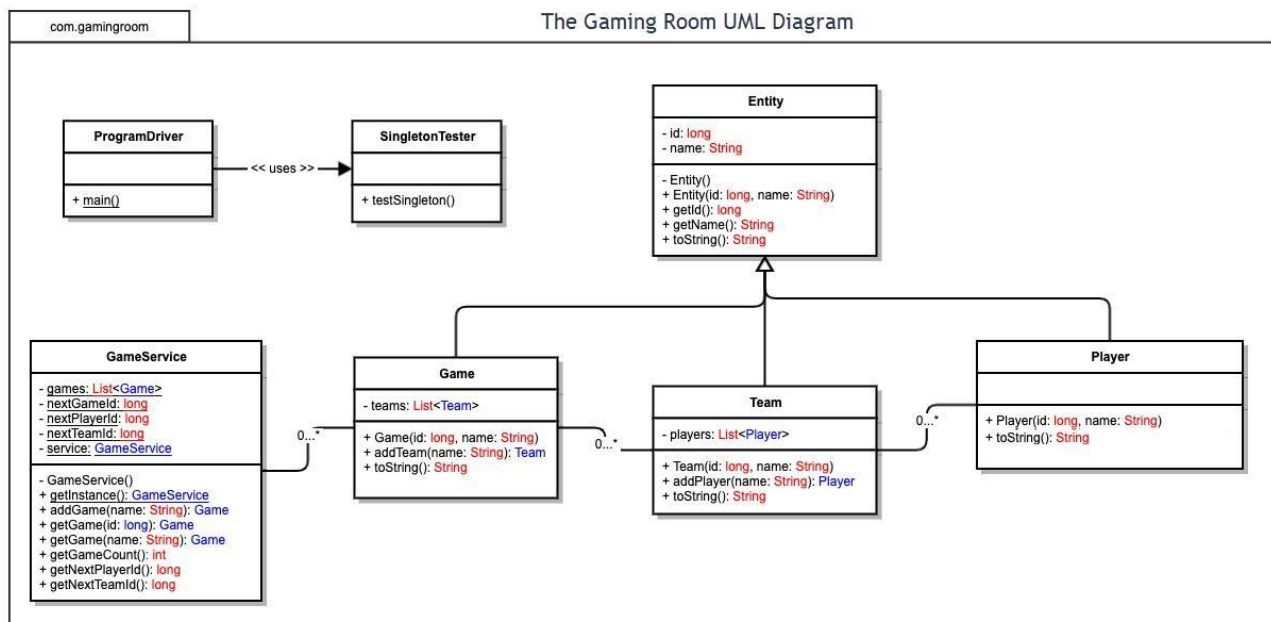
This design allows for instances controlled primarily through Inheritance and Aggregation. Using Inheritance, we see that the Game, Team and Player classes all inherit their attributes from Entity. This

relationship ensures all data is managed security and can be managed with a single set of security requirements.

Game and Team demonstrate Aggregation where Game contains Team objects. The multiplicity shows us a 0 to many relationships, implying we can have any number of teams in the game. Team meanwhile, has a zero to many relationships with Player. Meaning that a team may have multiple players in any single instance.

The ProgramDriver uses SingletonTester to verify the instances of the GameService. While GameService manages all objects within the List<Game>.

Using Inheritance with the Entity base class, we can control access to use and create unique game elements. Encapsulation within the game design supports legal and security requirements from outside methods or calls for information. The Singleton Pattern ensures only a single instance of a Game, Team or Player, ensuring resourcing constraint requirement are met (data and resource processing time).



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	While macOS offers a highly stable and secure operating system with known resistance to malware, it presents significant roadblocks for The Gaming Room's expansion goals. The primary disadvantages are the increased expense resulting from higher licensing costs. More critically, the platform is not designed for the efficient, large-scale distributed environment required, and it has a known weakness regarding application scalability. These factors, combined with limited software compatibility, render macOS an impractical choice for our server-side hosting solution.	Linux offers a highly cost-effective and structurally sound foundation for our application server. As an open-source platform, the primary advantage is the elimination of high operating system licensing fees, allowing resources to be shifted toward ensuring scalability. Linux is known for its stability, security, and reliability in hosting web applications, making it an excellent platform for our goal to scale this application. Additionally, its open-source nature promotes compatibility with the majority of required tools and platforms. The main challenges associated with Linux is its Command-Line Interface (CLI) which may require specialized expertise compared to the Graphical User Interface (GUI) of other platforms. An additional complication may arise from, the lack of standardization across devices. This is a mild concern compared to the licensing costs with other products.	Windows Server offers a robust infrastructure with native integration with the Microsoft Office suite. Its infrastructure provides a stable framework for web hosting and high security protocols. Its framework is capable of handling the demands of a scalable web-based application. The primary disadvantage is the significant price of licensing costs compared to open-source alternatives like Linux. These costs make Windows one of the least cost-effective options for server-side hosting. Additionally, system updates can occasionally lead to compatibility issues with various platforms or browsers. Additionally, this OS generally requires more hardware resources to run efficiently, adding to the total cost of use.	There are no practical advantages to using a mobile platform for server-side hosting; they excel only as client devices. Mobile devices are poorly suited to meet the requirements of a scalable web server. They are constrained by limited resources in terms of CPUs, RAM, and storage, directly compromising the speed and connectivity necessary for server operations. Their reliance on cellular data connections and their lack of security and administrative options leave them open to data breaches and attacks.

Client Side	Client-side development for MacOS requiring the use of a platform-agnostic framework is crucial to meet the requirement of a single codebase. This requirement is necessary to reduce the need for native MacOS expertise and allowing the development to be focused on the platform-agnostic framework. Time and cost considerations center on extensive testing required to ensure compatibility across multiple browsers and various OS versions. QA and testing time remains high to guarantee a positive user experience.	The use of a single codebase across all platforms ensures that the initial development time for Linux is comparable to macOS and Windows, as native Linux expertise is not required. This development approach maintains cost-effectiveness and consistency across the project work. A challenge unique to Linux environments will arise with QA, accompanied by a slight increase with costs. Linux open-source nature leads to a lack of standardization across various environments. Additionally, expertise is required to accommodate use of the Command-Line Interface (CLI) during testing and development.	Client-side development on the Windows platform will similarly rely on leveraging platform-agnostic codebase. Initial development time and expertise required are comparable to both macOS and Linux. The primary cost driver is the significant time investment required for QA testing. Testing must be planned across multiple OS versions and various browser versions. Additionally, Windows often requires specific hardware combinations unique to Windows that would require addition testing time and costs.	Client-side support for mobile devices) will be through the web browser, utilizing HTML interfaces. The key advantage of this is the avoidance of most native development costs associated with iOS and Android applications. This significantly lowers the initial investment costs that come from expertise and development time.
--------------------	---	---	--	--

Development Tools	Development on MacOS will require expertise in HTML5 and CSS for the user interface, JavaScript and/or TypeScript for front-end logic, and knowledge of the chosen platform-agnostic framework. The team will need to utilize cross-platform IDEs or similar multi-language tools. Additional tools include a runtime environment, a version control system, and browser-specific developer tools necessary for testing and troubleshooting OS integration. This approach prevents the need for multiple specialized native teams and minimizes licensing costs, as many core tools (VS Code, Node.js, GitHub) are free or open-source.	The development toolkit for Linux can be identical to that used for MacOS and Windows. This requires expertise in HTML5 and CSS for the user interface, JavaScript and/or TypeScript for front-end logic, and knowledge of the platform-agnostic framework. IDE support requires support for multi-language development, such. Additional necessary tools include a runtime environment, a version control program, and browser-specific development tools for cross-platform testing and troubleshooting. Utilizing these standard toolkits, ensures that a unified development team is enough to reduce costs in staffing, licensing and development costs.	The development languages are shared across all platforms, ensuring project consistency. This requires expertise in HTML5 and CSS for the user interface, JavaScript and/or TypeScript for front-end logic, and comprehensive knowledge of the platform-agnostic framework. The team will rely on many of the same multi-language development tools. While specialized tools for Windows development include code analysis tools. Additional essential tools include a front-end runtime environment, a version control, browser development tools, and, a test environment for QA across various Windows OS and hardware configurations. Toolsets are standardized between OS, with many being free, maintaining low project and licensing costs. A Single development team would be required and could focus resources on the specialized testing environment needed to validate compatibility across Windows environments.	Programming languages for mobile development are the same as other platforms. IDE tools still need a multi-platform development option, plus an emulator to test web interfaces. Testing and QA team members should include expertise for mobile app development to ensure proper function and development. Additional tools include a runtime platform and browser development tools, and an isolated testing environment.
--------------------------	--	--	--	--

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

Based on the above analysis for The Gaming Room's expansion of the Draw It or Lose It application, I recommend the following systems to ensure your project goals are met: scalability, security and cost-effectiveness.

1. **Operating Platform:** I recommend the Linux Operating System (OS) for the application's server environment. Linux offers the best combination of cost-effectiveness (due to minimal licensing fees), high security, reliability, and compatibility with the required development tools and operating systems.
 - a. Linux, as an open-source platform, Linux eliminates high licensing fees. Licensing fees constitutes a large portion of costs from other platforms. Eliminating licensing fees allows funds to be shifted as needed towards development and scalability, reducing overall costs and directly supporting the project's requirements to minimize overall costs.
 - b. Linux is known to be a stable, secure, reliable platform for hosting web applications. It's open source allows for a wide range of compatibility options both with other operating systems and with development tools. This combination allows for a dependable base platform for development.
2. **Operating Systems Architectures:** The application should be hosted using Linux as a Virtual Machine (VM). This provides essential benefits for scalability and resource management. Utilizing a VM assists with resource constraints while allowing efficient application hosting for the server.
 - a. Linux uses a monolithic kernel. This means the entire operating system operates within the same protected space. This minimizes server calls, and increases performance and efficiency. A high performance system is required to be able to handle multiple, simultaneous instances across platforms. A monolithic kernel also increases security by removing interactions between different systems when accessing core services such as memory, file management, device drivers and process communications.
 - b. Using a Virtual machine fills the project requirements for scalability and resource constraints. A Virtual Machine separates physical hardware from the operating system allowing for a simplification during application development.
3. **Storage Management:** I recommend using a cloud-based storage system with the VM. Many cost-effective storage systems are scalable and provide robust security and reliability. They typically include support options for game development and asset backup, ensuring minimal downtime and game interruptions compared to other services.
 - a. A cloud-based storage system in conjunction with a VM ensures data integrity is maintained throughout the life of the application. This becomes especially important with reversioning and protection against outages and system failures.
 - b. Cloud storage is cost effective and scaleable. Most cloud-based storage plans include an additional security layer of their own, as well as support for system back-ups and development.
4. **Memory Management:** Memory management within Linux VM is crucial for system stability and resource management.

- a. The system achieves process isolation by assigning the game its own unique address via the kernel, maintaining security and stability in its use of a single instance of the game, utilizing a Singleton Pattern to facilitate game play.
 - b. This process may be further divided into smaller units, (pages), which allows for calls to be made to specific units as needed, ensuring data is kept safe and improving overall efficiency.
 - c. Memory is allocated to a shared library system, which hosts common elements like the Singleton Manager. This practice reduces the need for duplicate code and overall memory consumption.
- 5. **Distributed Systems and Networks:** The application utilizes a three-tiered distributed system to manage communication and data flow across multiple platforms:
 - a. Client Tier: The user interface runs on a standard web browser and platforms, facilitating communication with the application server via secure protocols, such as HTTPS.
 - b. Application Tier: The application server runs on a Linux-based VM. It hosts the game, its elements, and instances, managed efficiently via the Singleton Manager (GameService).
 - c. Data Tier: The application manager communicates with a cloud-based data storage center to securely store all current and historical data.
 - Communication between these systems and applications must be fast and reliable for optimal gameplay, creating a dependency on the cloud server and network speeds.
 - To minimize disruptions that can impact data integrity and reliability, network calls should be limited as much as possible, including combining elements into a single call when feasible.
- 6. **Security:** Security requirements must be maintained across the design through integrated processes and structures.
 - a. To ensure security and privacy are maintained, all applications must be integrated and authenticated using trusted third-party processes. Performing this step early ensures the encapsulation process supports legal and data privacy laws.
 - b. Access to gameplay and user information is strictly controlled using unique identifiers that can only be accessed through the inheritance structures built from the Entity Class. This ensures that security is maintained and controlled throughout the application.
 - c. The Linux Operating system, cloud storage, and all systems and programs within this project will utilize standard measures such as data encryption, firewalls, and built-in security protocols to maintain the confidentiality of all data sources.